

Full Report:

===== KFA LIBRARY CATALOGUE REPORT =====

- Clean Code [Status: Borrowed] | Lending Days: 14
- Tech Monthly [Status: Available] | Lending Days: 7
- Inception [Status: Available] | Lending Days: 5
- Effective Java [Status: Available] | Lending Days: 14

Filtered Report (Keyword 'java'):

===== KFA LIBRARY CATALOGUE REPORT =====

- Effective Java [Status: Available] | Lending Days: 14

Polymorphism and Hierarchy

[9780132350884] Clean Code Rs 850.00 (Available)

Lending Period: 14 days

[Magazine #42] Tech Monthly Rs 250.00 (Available)

Lending Period: 7 days

[DVD 148 mins] Inception Rs 500.00 (Available)

Lending Period: 5 days

[9780134685991] Effective Java Rs 1200.00 (Available)

Lending Period: 14 days

Custom Exception Handling

Successfully borrowed: Clean Code

Transaction processed for: Clean Code

BORROW ERROR: The item 'Clean Code' is currently out on loan.

Transaction processed for: Clean Code

RETURN ERROR: Item is overdue by 3 day(s).

Transaction processed for: Tech Monthly

String Utilities & StringBuilder

Member ID: AARSH872

Member ID (Single Name): PLAT0665

ISBN Validation Tests:

ISBN 9780132350884 -> Valid: true

ISBN 0123456789012 -> Valid: false

ISBN 978013235088 -> Valid: false

ISBN 978013235088X -> Valid: false

RFA. model.

* DVD.java.

public class DVD extends LibraryItem implements Renewable {
 private int duration minutes;

public DVD (String title, String isbn, double price, int duration
 minutes) {
 super (title, isbn, price);

this.duration minutes = duration minutes;

}

public int getDuration minutes () {
 return duration minutes;

}

@Override

public int getLending Period days () {
 return 5;

}

@Override

public void renew (int extra days) {
 System.out.println ("DVD " + getTitle () + " renewed for an
 additional " + extra days + " days");

}

@Override

public String toString () {

return "[DVD " + duration minutes + " mins] " + getTitle () +

"Rs " + String.format ("%2f", getPrice ()) + " (" + (is Available ()

? "Available" : "Borrowed") + ")";

}

}

KFA. exception.

// BookNotAvailable Exception;

Public class BookNotAvailableException extends Exception {

Public BookNotAvailableException (storing message) {

super (message);

}

}

// ItemOverdue Exception.

Public class ItemOverdueException extends Exception {

Private int daysOverdue ;

Public ItemOverdueException (int dayOver due) {

super (" Item is overdue by " + daysOverdue + " day(s). ");

this .daysOverdue = daysOverdue;

}

Public int getDaysOverdue() {

return daysOverdue ;

}

KFA. model

* Book.java.

```
public class Book extends Library Items, implements Renewable {  
    private static int totalBooks = 0;
```

```
    public Book (String title, String isbn, double price) {  
        super (title, isbn, price);  
        totalBooks ++;
```

```
    }
```

```
    public static int getTotalBooks () {  
        return totalBooks;  
    }
```

@Override

```
    public int getLendingPeriodDays () {  
        return 14;
```

```
    }
```

@Override

```
    public void renew (int extraDays) {  
        System.out.println ("Book " + getTitle () + " renewed  
        for an additional " + extraDays + " days. ");
```

```
    }
```

@Override

```
    public String toString () {  
        return "[" + getIsbn () + "]" + getTitle () + " Rs " +  
            String.format ("%.2f", getPrice ()) + " (" +  
            (isAvailable () ? "Available" : "Borrowed") + ")";
```


KFA model.

* LibraryItems.java

Public abstract class LibraryItems{

Private String title;

Private String isbn;

Private double price;

Private boolean available;

Public LibraryItem(String title, String isbn, double price){

this.title = title;

this.isbn = isbn;

~~this~~.setPrice(price);

this.available = true;

}

Public abstract int getLendingPeriodDays();

Public String getTitle() { return title; }

Public String getIsbn() { return isbn; }

Public double getPrice() { return price; }

Public boolean isAvailable() { return available; }

Public void setPrice(double price){

if (price >= 0){

• this.price = price;

}

else {
System.out.println("Invalid Price. Price can't be negative");

this.price = 0.0;

}

}

}



```

public String buildCatalogueReport (LibraryItem[] items,
String filterKeyword) {
    String Builder sb = new String Builder();
    sb.append("KFA LIBRARY CATALOGUE REPORT");
    Boolean hasKeyword = (filterKeyword != null && !filterKeyword.
    trim().isEmpty());
    String search = hasKeyword ? filterKeyword.trim().toLowerCase() :
    "";

```

```

    for (LibraryItem item : items) {
        if (!hasKeyword || item.getTitle().toLowerCase().
        contains(search)) {
            sb.append("- ").append(item.getTitle())
            .append("[ Status: ").append(item.isAvailable() ?
            "Available" : "Borrowed").append("] ")
            .append("/ Lending Days: ").append(item.getLending
            periodDays())
            .append("\n");

```

```

        }
    }
    return sb.toString();
}
}

```



```

else {
    String firstName = posts[0].toUpperCase();
    String lastName = posts[posts.length - 1].toUpperCase();
    firstPost = firstName.length() >= 3 ? firstName.substring(0, 3) :
        firstName;
    lastPost = lastName.length() >= 2 ? lastName.substring(0, 2) :
        lastName;
}
int randomNum = new Random().nextInt(900) + 100;
return firstPost + lastPost + randomNum;
}

```

```

}
public static boolean isValidIsbn(String isbn) {
    if (isbn == null || isbn.length() != 13) {
        return false;
    }
    if (isbn.charAt(0) == "0") {
        return false;
    }
    for (int i = 0; i < isbn.length(); i++) {
        if (!Character.isDigit(isbn.charAt(i))) {
            return false;
        }
    }
}

```

```

return true;
}

```

KFA.service
* LibrarySystem.

import KFA.exception.*;
import KFA.model.*;
import ~~KFA~~ java.util.Random;

Public class LibrarySystem {

Public void borrowItem(LibraryItem item) throws
BookNotAvailableException {

if (!item.isAvailable()) {

throw new BookNotAvailableException("The item." +
item.getTitle() + " is currently out on loan.");

}

item.setAvailable(false);

}

Public void returnItem(LibraryItem, int daysLate) throws
ItemOverdueException {

if (daysLate > 0) {

throw new ItemOverdueException(daysLate);

}

item.setAvailable(true);

}

Public static String generateMemberId(String fullName) {

if (fullName == null) return " ";

String formatted = fullName.trim();

String[] parts = formatted.split(" ");

String firstPart = " ";

String lastPart = " ";

if (parts.length == 1) {

String single = parts[0].toUpperCase();

firstPart = single.length() > 3 ? single.substring(0, 3) : single;

lastPart = single.length() > 5 ? single.substring(3, 5) : "xx";

}

```
for (String isbn : testIsbns) {  
    system.out.println ("ISBN " + isbn + " -> Valid" +  
        LibrarySystem.isValidIsbn (isbn));  
}
```

```
for (String isbn : testIsbn) {  
    system.out.println ("ISBN " + isbn + " -> valid " +  
        i.  
}
```

```
private static void runBorrowTransaction(LibrarySystem, Library  
Items) {
```

```
    try {  
        system.borrowsItems (items);  
        system.out.println ("successfully borrowed " + item.getTitle());  
    }
```

```
    catch (BookNotAvailableException e) {  
        system.out.println ("Borrow error: " + e.getMessage());  
    }
```

```
    finally {  
        system.out.println ("Transaction process for: " + item.getTitle());  
    }
```

```
    }  
}
```


* main.java

```
import KFA.model.*;  
import KFA.model.exception.*;  
import KFA.service.*;
```

Public class main {

```
    public static void main (String[] args) {  
        System.out.println("Polymorphism & Inheritance");  
        LibraryItems[] items = new LibraryItems[3];  
        new Book ("Class Code", "9780132350884", 850.00),  
        new Magazine ("Tech monthly", "978...", 250, 42),  
        new DVD ("Inception", "978...", 500, 148),  
        new Book ("Effective Java", "978...", 1200)  
    };
```

for (LibraryItem item : items) {

```
    System.out.println (item.toString());  
    System.out.println ("Lending period: " + item.getLending  
        PeriodDays() + " days\n");  
}
```

```
    System.out.println ("Total Book created: " + Book.getTotalBooks());
```

```
    System.out.println ("\n Custom Exception Handling ");
```

```
    LibrarySystem = new LibrarySystem();
```

```
    runBorrowTransactions (system, items[0]);
```

```
    System.out.println ("string utilization & string Builder");
```

```
    System.out.println ("number 12: " + LibrarySystem.generateMember  
        Id ("Aman sharma"));
```

```
    System.out.println ("member ID: " + LibrarySystem.generateMemberId  
        ("prab"));
```

```
    System.out.println ("\n ISBN Validation Tests: ");
```

```
    String[] testISBNs = {  
        ".....",  
        "98....",  
        "978....." };
```